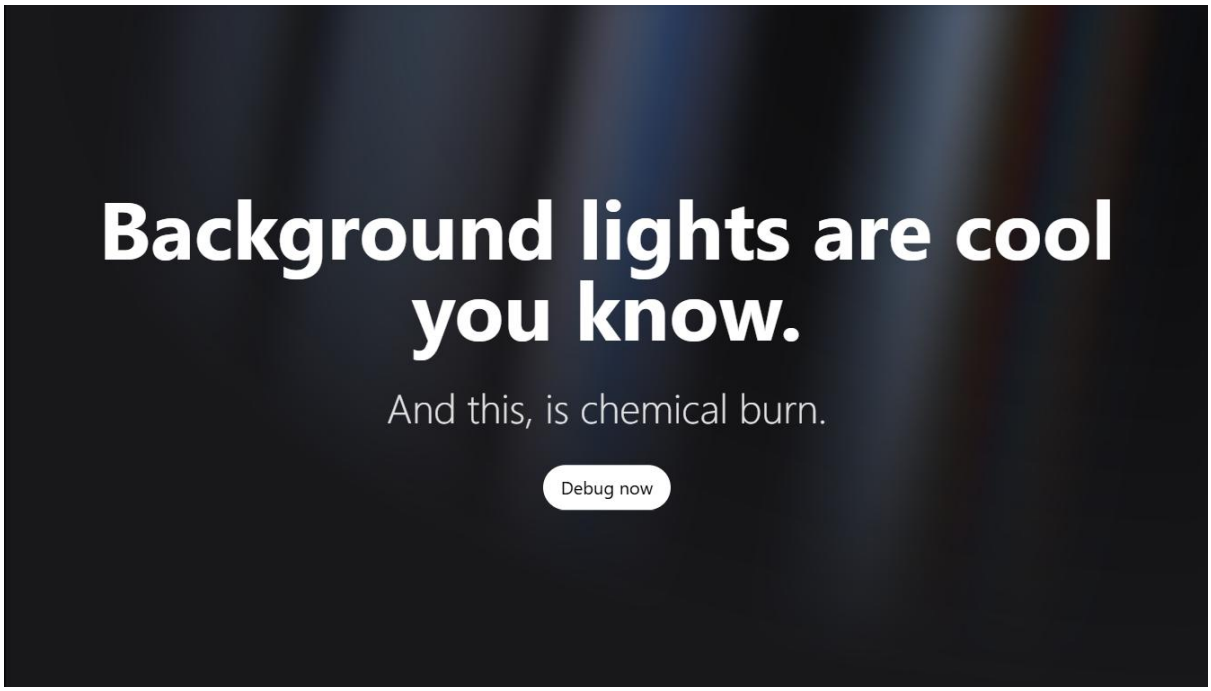
```
"use client";
import { cn } from "@/lib/utils";
import React, { ReactNode } from "react";

interface AuroraBackgroundProps extends React.HTMLProps<HTMLDivElement> {
  children: ReactNode;
  showRadialGradient?: boolean;
}

export const AuroraBackground = ({
  className,
  children,
  showRadialGradient = true,
  ...props
}: AuroraBackgroundProps) => {
  return (
    <main>
      <div
        className={cn(
          "relative flex flex-col  h-[100vh] items-center justify-center bg-zinc-50 dark:bg-zinc-900 text-slate-950 transition-bg",
          className
        )}
        {...props}
      >
        <div className="absolute inset-0 overflow-hidden">
          <div
            //   I'm sorry but this is what peak developer performance looks like // trigger warning
```

```
        className={cn(
          `
        [--white-gradient:repeating-linear-gradient(100deg,var(--white)_0%,var(--white)_7%,var(--transparent)_10%,var(--transparent)_12%,var(--white)_16%)]
        [--dark-gradient:repeating-linear-gradient(100deg,var(--black)_0%,var(--black)_7%,var(--transparent)_10%,var(--transparent)_12%,var(--black)_16%)]
        [--aurora:repeating-linear-gradient(100deg,var(--blue-500)_10%,var(--indigo-300)_15%,var(--blue-300)_20%,var(--violet-200)_25%,var(--blue-400)_30%)]
        [background-image:var(--white-gradient),var(--aurora)]
        dark:[background-image:var(--dark-gradient),var(--aurora)]
        [background-size:300%,_200%]
        [background-position:50%_50%,50%_50%]
        filter blur-[10px] invert dark:invert-0
        after:content-[""] after:absolute after:inset-0 after:[background-image:var(--white-gradient),var(--aurora)]
        after:dark:[background-image:var(--dark-gradient),var(--aurora)]
        after:[background-size:200%,_100%]
        after:animate-aurora after:[background-attachment:fixed] after:mix-blend-difference
        pointer-events-none
        absolute -inset-[10px] opacity-50 will-change-transform`,

          showRadialGradient &&
            `[mask-image:radial-gradient(ellipse_at_100%_0%,black_10%,var(--transparent)_70%)]`
        )}
       ></div>
      </div>
      {children}
    </div>
```

```
    </main>
  );
};
```

It also contains this type of reveal text:

1.On the Landing Page:



Code:

```
"use client";

import { motion } from "framer-motion";
import { useState, useEffect } from "react";

interface RevealTextProps {
  text?: string;
```

```
  textColor?: string;
  overlayColor?: string;
  fontSize?: string;
  letterDelay?: number;
  overlayDelay?: number;
  overlayDuration?: number;
  springDuration?: number;
  letterImages?: string[];
}

export function RevealText({
  text = "STUNNING",
  textColor = "text-white",
  overlayColor = "text-red-500",
  fontSize = "text-[250px]",
  letterDelay = 0.08,
  overlayDelay = 0.05,
  overlayDuration = 0.4,
  springDuration = 600,
  letterImages = [
    "https://images.unsplash.com/photo-1506905925346-21bda4d32df4?ixlib=rb-4.0.3&auto=format&fit=crop&w=2070&q=80", // S
    "https://images.unsplash.com/photo-1469474968028-56623f02e42e?ixlib=rb-4.0.3&auto=format&fit=crop&w=2070&q=80", // T
    "https://images.unsplash.com/photo-1518837695005-2083093ee35b?ixlib=rb-4.0.3&auto=format&fit=crop&w=2070&q=80", // U
    "https://images.unsplash.com/photo-1501594907352-04cda38ebc29?ixlib=rb-4.0.3&auto=format&fit=crop&w=2070&q=80", // N
```

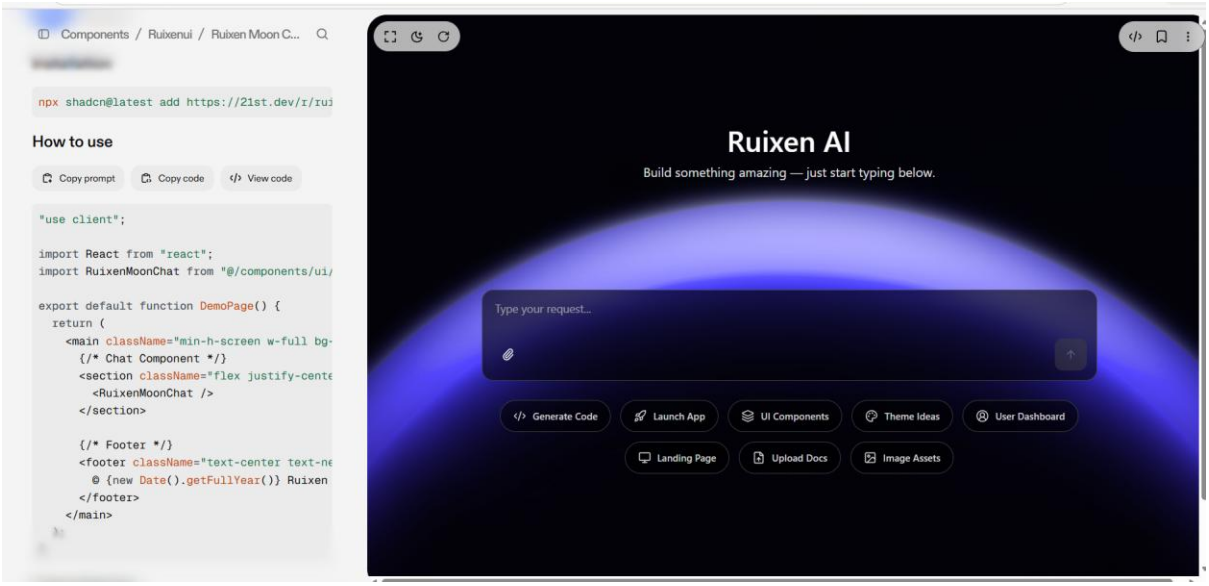
```
    "https://images.unsplash.com/photo-1472214103451-9374bd1c798e?ixlib=rb-4.0.3&auto=format&fit=crop&w=2070&q=80", // N
    "https://images.unsplash.com/photo-1441974231531-c6227db76b6e?ixlib=rb-4.0.3&auto=format&fit=crop&w=2070&q=80", // I
    "https://images.unsplash.com/photo-1519904981063-b0cf448d479e?ixlib=rb-4.0.3&auto=format&fit=crop&w=2070&q=80", // N
    "https://images.unsplash.com/photo-1540979388789-6cee28a1cdc9?ixlib=rb-4.0.3&auto=format&fit=crop&w=2070&q=80", // G
  ]
}: RevealTextProps) {
  const [hoveredIndex, setHoveredIndex] = useState<number | null>(null);
  const [showRedText, setShowRedText] = useState(false);

  useEffect(() => {
    // Calculate when the last letter animation completes
    // Last letter starts at (text.length - 1) * letterDelay seconds
    // Add springDuration for the spring animation to settle
    const lastLetterDelay = (text.length - 1) * letterDelay;
    const totalDelay = (lastLetterDelay * 1000) + springDuration;

    const timer = setTimeout(() => {
      setShowRedText(true);
    }, totalDelay);

    return () => clearTimeout(timer);
  }, [text.length, letterDelay, springDuration]);

  return (
    <div className="flex items-center justify-center relative">
```

```
    <div className="flex">
      {text.split("").map((letter, index) => (
        <motion.span
          key={index}
          onMouseEnter={() => setHoveredIndex(index)}
          onMouseLeave={() => setHoveredIndex(null)}
          className={`${fontSize} font-black tracking-tight cursor-pointer relative overflow-hidden`}
          initial={{
            scale: 0,
            opacity: 0,
          }}
          animate={{
            scale: 1,
            opacity: 1,
          }}
          transition={{
            delay: index * letterDelay,
            type: "spring",
            damping: 8,
            stiffness: 200,
            mass: 0.8,
          }}
        >
          {/* Base text layer */}
          <motion.span
            className={`absolute inset-0 ${textColor}`}
            animate={{
```

```
    opacity: hoveredIndex === index ? 0 : 1
  }}
  transition={{ duration: 0.1 }}
>
  {letter}
</motion.span>
{/* Image text layer with background panning */}
<motion.span
  className="text-transparent bg-clip-text bg-cover bg-no-repeat"
  animate={{
    opacity: hoveredIndex === index ? 1 : 0,
    backgroundPosition: hoveredIndex === index ? "10% center" : "0% center"
  }}
  transition={{
    opacity: { duration: 0.1 },
    backgroundPosition: {
      duration: 3,
      ease: "easeInOut"
    }
  }}
  style={{
    backgroundImage: `url('${letterImages[index % letterImages.length]}')`,
    WebkitBackgroundClip: "text",
    WebkitTextFillColor: "transparent",
  }}
>
  {letter}
</motion.span>
```

```
          {/* Overlay text layer that sweeps across each letter */}
          {showRedText && (
            <motion.span
              className={`absolute inset-0 ${overlayColor} pointer-events-none`}
              initial={{ opacity: 0 }}
              animate={{
                opacity: [0, 1, 1, 0]
              }}
              transition={{
                delay: index * overlayDelay,
                duration: overlayDuration,
                times: [0, 0.1, 0.7, 1],
                ease: "easeInOut"
              }}
            >
              {letter}
            </motion.span>
          )}
        </motion.span>
      ))}
    </div>
  </div>
 );
}
```

2.For chatbot background and chatbox which takes the input for AI:




```
"use client";

import { useState, useRef, useEffect, useCallback } from "react";
import { Textarea } from "@/components/ui/textarea";
import { Button } from "@/components/ui/button";
import { cn } from "@/lib/utils";
import {
  ImageIcon,
  FileUp,
  MonitorIcon,
  CircleUserRound,
  ArrowUpIcon,
  Paperclip,
  PlusIcon,
  Code2,
  Palette,
  Layers,
```

```
  Rocket,
} from "lucide-react";

interface AutoResizeProps {
  minHeight: number;
  maxHeight?: number;
}

function useAutoResizeTextarea({ minHeight, maxHeight }: AutoResizeProps) {
  const textareaRef = useRef<HTMLTextAreaElement>(null);

  const adjustHeight = useCallback(
    (reset?: boolean) => {
      const textarea = textareaRef.current;
      if (!textarea) return;

      if (reset) {
        textarea.style.height = `${minHeight}px`;
        return;
      }

      textarea.style.height = `${minHeight}px`; // reset first
      const newHeight = Math.max(
        minHeight,
        Math.min(textarea.scrollHeight, maxHeight ?? Infinity)
      );
      textarea.style.height = `${newHeight}px`;
    },
```

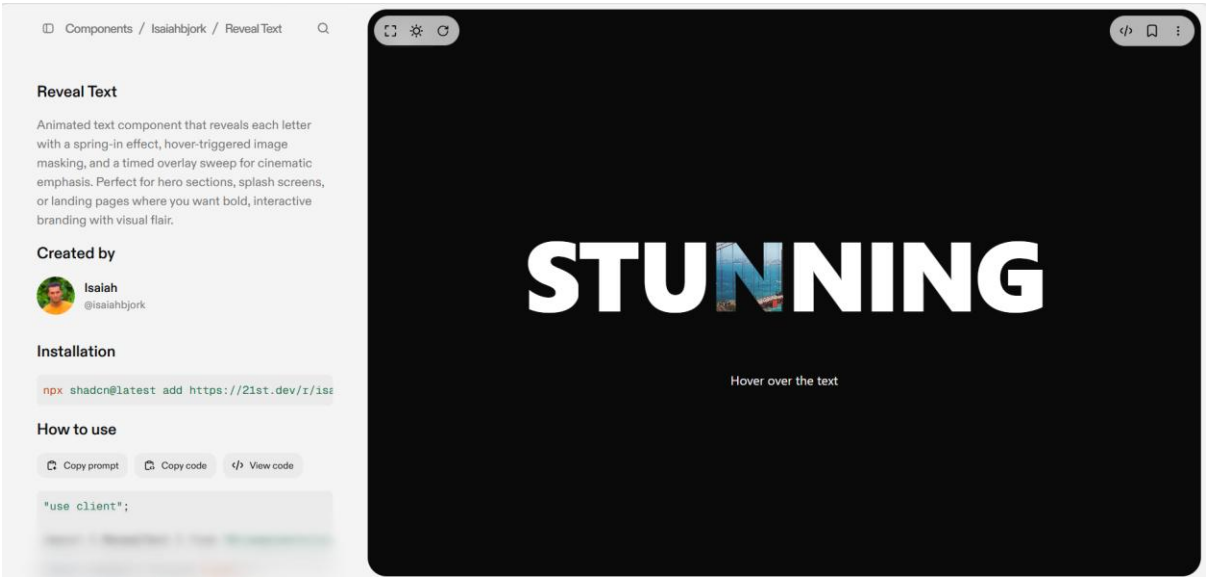
```
    [minHeight, maxHeight]
  );

  useEffect(() => {
    if (textareaRef.current) textareaRef.current.style.height = `${minHeight}px`;
  }, [minHeight]);

  return { textareaRef, adjustHeight };
}

export default function RuixenMoonChat() {
  const [message, setMessage] = useState("");
  const { textareaRef, adjustHeight } = useAutoResizeTextarea({
    minHeight: 48,
    maxHeight: 150,
  });

  return (
    <div
      className="relative w-full h-screen bg-cover bg-center flex flex-col items-center"
      style={{
        backgroundImage:
          "url('https://pub-940ccf6255b54fa799a9b01050e6c227.r2.dev/ruixen_moon_2.png')",
        backgroundAttachment: "fixed",
      }}
    >
      {/* Centered AI Title */}
      <div className="flex-1 w-full flex flex-col items-center justify-center">
```

```
        <div className="text-center">
          <h1 className="text-4xl font-semibold text-white drop-shadow-sm">
            Ruixen AI
          </h1>
          <p className="mt-2 text-neutral-200">
            Build something amazing — just start typing below.
          </p>
        </div>
      </div>

      {/* Input Box Section */}
      <div className="w-full max-w-3xl mb-[20vh]">
        <div className="relative bg-black/60 backdrop-blur-md rounded-xl border border-neutral-700">
          <Textarea
            ref={textareaRef}
            value={message}
            onChange={(e) => {
              setMessage(e.target.value);
              adjustHeight();
            }}
            placeholder="Type your request..."
            className={cn(
              "w-full px-4 py-3 resize-none border-none",
              "bg-transparent text-white text-sm",
              "focus-visible:ring-0 focus-visible:ring-offset-0",
              "placeholder:text-neutral-400 min-h-[48px]"
            )}
```

```
  style={{ overflow: "hidden" }}
 />

 {/* Footer Buttons */}
 <div className="flex items-center justify-between p-3">
  <Button
   variant="ghost"
   size="icon"
   className="text-white hover:bg-neutral-700"
  >
   <Paperclip className="w-4 h-4" />
  </Button>

  <div className="flex items-center gap-2">
   <Button
    disabled
    className={cn(
     "flex items-center gap-1 px-3 py-2 rounded-lg transition-colors",
     "bg-neutral-700 text-neutral-400 cursor-not-allowed"
    )}
   >
    <ArrowUpIcon className="w-4 h-4" />
    <span className="sr-only">Send</span>
   </Button>
  </div>
 </div>
</div>
```

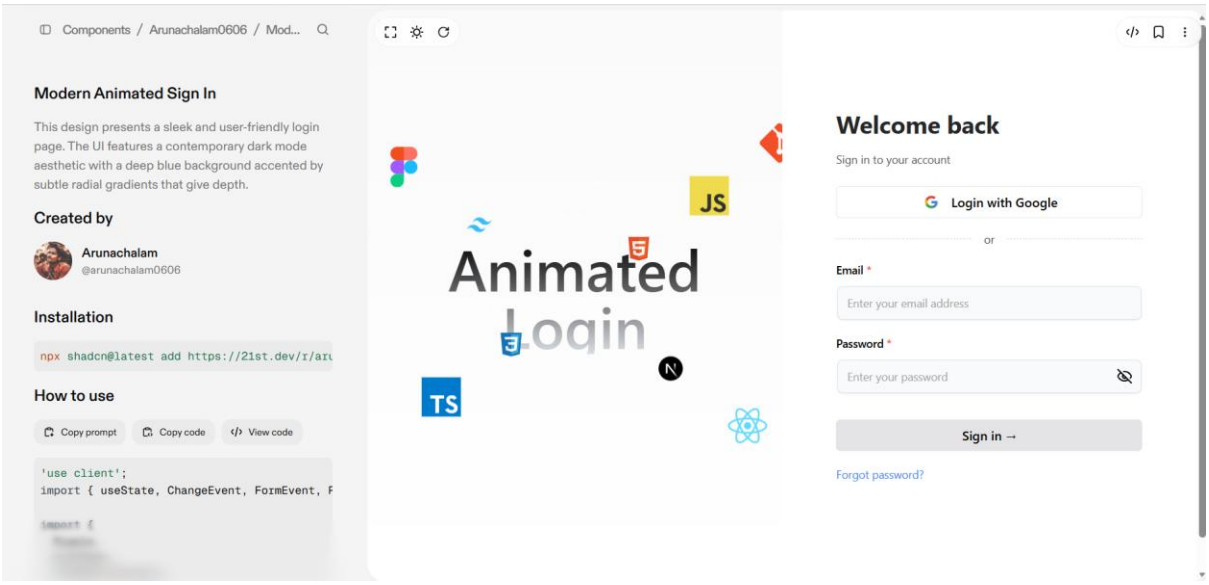
```
      {/* Quick Actions */}
      <div className="flex items-center justify-center flex-wrap gap-3 mt-6">
        <QuickAction icon={<Code2 className="w-4 h-4" />} label="Generate Code" />
        <QuickAction icon={<Rocket className="w-4 h-4" />} label="Launch App" />
        <QuickAction icon={<Layers className="w-4 h-4" />} label="UI Components" />
        <QuickAction icon={<Palette className="w-4 h-4" />} label="Theme Ideas" />
        <QuickAction icon={<CircleUserRound className="w-4 h-4" />} label="User Dashboard" />
        <QuickAction icon={<MonitorIcon className="w-4 h-4" />} label="Landing Page" />
        <QuickAction icon={<FileUp className="w-4 h-4" />} label="Upload Docs" />
        <QuickAction icon={<ImageIcon className="w-4 h-4" />} label="Image Assets" />
      </div>
    </div>
  </div>
 );
}
```

3.For landing page text:



```
interface QuickActionProps {
  icon: React.ReactNode;
  label: string;
}

function QuickAction({ icon, label }: QuickActionProps) {
  return (
    <Button
      variant="outline"
      className="flex items-center gap-2 rounded-full border-neutral-700 bg-black/50 text-neutral-300 hover:text-white hover:bg-neutral-700"
    >
      {icon}
      <span className="text-xs">{label}</span>
    </Button>
  );
}
```

4.For Login Page having this complete page as a design:



THe main text should be changed to the Saas Website name instead of Animated login

```
'use client';
import {
  memo,
  ReactNode,
  useState,
  ChangeEvent,
  FormEvent,
  useEffect,
  useRef,
  forwardRef,
} from 'react';
import Image from 'next/image';
import {
  motion,
  useAnimation,
  useInView,
  useMotionTemplate,
```

```
  useMotionValue,
} from 'motion/react';
import { Eye, EyeOff } from 'lucide-react';
import { cn } from '@/lib/utils';

// ==================== Input Component ====================

const Input = memo(
  forwardRef(function Input(
    { className, type, ...props }: React.InputHTMLAttributes<HTMLInputElement>,
    ref: React.ForwardedRef<HTMLInputElement>
  ) {
    const radius = 100; // change this to increase the radius of the hover effect
    const [visible, setVisible] = useState(false);

    const mouseX = useMotionValue(0);
    const mouseY = useMotionValue(0);

    function handleMouseMove({
      currentTarget,
      clientX,
      clientY,
    }: React.MouseEvent<HTMLDivElement>) {
      const { left, top } = currentTarget.getBoundingClientRect();

      mouseX.set(clientX - left);
      mouseY.set(clientY - top);
    }
```

```
  return (
    <motion.div
      style={{
        background: useMotionTemplate`
      radial-gradient(
        ${visible ? radius + 'px' : '0px'} circle at ${mouseX}px ${mouseY}px,
        #3b82f6,
        transparent 80%
      )
    `,
      }}
      onMouseMove={handleMouseMove}
      onMouseEnter={() => setVisible(true)}
      onMouseLeave={() => setVisible(false)}
      className='group/input rounded-lg p-[2px] transition duration-300'
    >
      <input
        type={type}
        className={cn(
          `shadow-input dark:placeholder-text-neutral-600 flex h-10 w-full rounded-md border-none bg-gray-50 px-3 py-2 text-sm text-black transition duration-400 group-hover/input:shadow-none file:border-0 file:bg-transparent file:text-sm file:font-medium placeholder:text-neutral-400 focus-visible:ring-[2px] focus-visible:ring-neutral-400 focus-visible:outline-none disabled:cursor-not-allowed disabled:opacity-50 dark:bg-zinc-800 dark:text-white dark:shadow-[0px_0px_1px_1px_#404040] dark:focus-visible:ring-neutral-600`,
          className
        )}
        ref={ref}
```

```
        {...props}
      />
    </motion.div>
  );
 })
);

Input.displayName = 'Input';

// ==================== BoxReveal Component ====================

type BoxRevealProps = {
 children: ReactNode;
 width?: string;
 boxColor?: string;
 duration?: number;
 overflow?: string;
 position?: string;
 className?: string;
};

const BoxReveal = memo(function BoxReveal({
 children,
 width = 'fit-content',
 boxColor,
 duration,
 overflow = 'hidden',
 position = 'relative',
```

```
  className,
}: BoxRevealProps) {
  const mainControls = useAnimation();
  const slideControls = useAnimation();
  const ref = useRef(null);
  const isInView = useInView(ref, { once: true });

  useEffect(() => {
    if (isInView) {
      slideControls.start('visible');
      mainControls.start('visible');
    } else {
      slideControls.start('hidden');
      mainControls.start('hidden');
    }
  }, [isInView, mainControls, slideControls]);

  return (
    <section
      ref={ref}
      style={{
        position: position as
          | 'relative'
          | 'absolute'
          | 'fixed'
          | 'sticky'
          | 'static',
        width,
```

```
    overflow,
  }}
  className={className}
>
  <motion.div
    variants={{
      hidden: { opacity: 0, y: 75 },
      visible: { opacity: 1, y: 0 },
    }}
    initial='hidden'
    animate={mainControls}
    transition={{ duration: duration ?? 0.5, delay: 0.25 }}
  >
    {children}
  </motion.div>
  <motion.div
    variants={{ hidden: { left: 0 }, visible: { left: '100%' } }}
    initial='hidden'
    animate={slideControls}
    transition={{ duration: duration ?? 0.5, ease: 'easeIn' }}
    style={{
      position: 'absolute',
      top: 4,
      bottom: 4,
      left: 0,
      right: 0,
      zIndex: 20,
      background: boxColor ?? '#5046e6',
```

```
        borderRadius: 4,
      }}
    />
  </section>
 );
});

// ==================== Ripple Component ====================

type RippleProps = {
 mainCircleSize?: number;
 mainCircleOpacity?: number;
 numCircles?: number;
 className?: string;
};

const Ripple = memo(function Ripple({
 mainCircleSize = 210,
 mainCircleOpacity = 0.24,
 numCircles = 11,
 className = '',
}: RippleProps) {
 return (
  <section
   className={`max-w-[50%] absolute inset-0 flex items-center justify-center
    dark:bg-white/5 bg-neutral-50
    [mask-image:linear-gradient(to_bottom,black,transparent)]
    dark:[mask-image:linear-gradient(to_bottom,white,transparent)] ${className}`}
```

```
>
  {Array.from({ length: numCircles }, (_, i) => {
    const size = mainCircleSize + i * 70;
    const opacity = mainCircleOpacity - i * 0.03;
    const animationDelay = `${i * 0.06}s`;
    const borderStyle = i === numCircles - 1 ? 'dashed' : 'solid';
    const borderOpacity = 5 + i * 5;

    return (
      <span
        key={i}
        className='absolute animate-ripple rounded-full bg-foreground/15 border'
        style={{
          width: `${size}px`,
          height: `${size}px`,
          opacity: opacity,
          animationDelay: animationDelay,
          borderStyle: borderStyle,
          borderWidth: '1px',
          borderColor: `var(--foreground) dark:var(--background) / ${
            borderOpacity / 100
          })`,
          top: '50%',
          left: '50%',
          transform: 'translate(-50%, -50%)',
        }}
      />
    );
```

```
      })}
    </section>
  );
});

// ==================== OrbitingCircles Component ====================

type OrbitingCirclesProps = {
  className?: string;
  children: ReactNode;
  reverse?: boolean;
  duration?: number;
  delay?: number;
  radius?: number;
  path?: boolean;
};

const OrbitingCircles = memo(function OrbitingCircles({
  className,
  children,
  reverse = false,
  duration = 20,
  delay = 10,
  radius = 50,
  path = true,
}: OrbitingCirclesProps) {
  return (
    <>
```

```
{path && (
  <svg
    xmlns='http://www.w3.org/2000/svg'
    version='1.1'
    className='pointer-events-none absolute inset-0 size-full'
  >
    <circle
      className='stroke-black/10 stroke-1 dark:stroke-white/10'
      cx='50%'
      cy='50%'
      r={radius}
      fill='none'
    />
  </svg>
)}
<section
  style={
    {
      '--duration': duration,
      '--radius': radius,
      '--delay': -delay,
    } as React.CSSProperties
  }
  className={cn(
    'absolute flex size-full transform-gpu animate-orbit items-center justify-center rounded-full border bg-black/10 [animation-delay:calc(var(--delay)*1000ms)] dark:bg-white/10',
    { '[animation-direction:reverse]': reverse },
    className
```

```
        )}
      >
        {children}
      </section>
    </>
  );
});

// ==================== TechOrbitDisplay Component ====================

type IconConfig = {
  className?: string;
  duration?: number;
  delay?: number;
  radius?: number;
  path?: boolean;
  reverse?: boolean;
  component: () => React.ReactNode;
};

type TechnologyOrbitDisplayProps = {
  iconsArray: IconConfig[];
  text?: string;
};

const TechOrbitDisplay = memo(function TechOrbitDisplay({
  iconsArray,
  text = 'Animated Login',
```

```
}: TechnologyOrbitDisplayProps) {
  return (
    <section className='relative flex h-full w-full flex-col items-center justify-center overflow-hidden rounded-lg'>
      <span className='pointer-events-none whitespace-pre-wrap bg-gradient-to-b from-black to-gray-300/80 bg-clip-text text-center text-7xl font-semibold leading-none text-transparent dark:from-white dark:to-slate-900/10'>
        {text}
      </span>

      {iconsArray.map((icon, index) => (
        <OrbitingCircles
          key={index}
          className={icon.className}
          duration={icon.duration}
          delay={icon.delay}
          radius={icon.radius}
          path={icon.path}
          reverse={icon.reverse}
        >
          {icon.component()}
        </OrbitingCircles>
      ))}
    </section>
  );
});

// ==================== AnimatedForm Component ====================
```

```
type FieldType = 'text' | 'email' | 'password';

type Field = {
  label: string;
  required?: boolean;
  type: FieldType;
  placeholder?: string;
  onChange: (event: ChangeEvent<HTMLInputElement>) => void;
};

type AnimatedFormProps = {
  header: string;
  subHeader?: string;
  fields: Field[];
  submitButton: string;
  textVariantButton?: string;
  errorField?: string;
  fieldPerRow?: number;
  onSubmit: (event: FormEvent<HTMLFormElement>) => void;
  googleLogin?: string;
  goTo?: (event: React.MouseEvent<HTMLButtonElement>) => void;
};

type Errors = {
  [key: string]: string;
};

const AnimatedForm = memo(function AnimatedForm({
```

```
  header,
  subHeader,
  fields,
  submitButton,
  textVariantButton,
  errorField,
  fieldPerRow = 1,
  onSubmit,
  googleLogin,
  goTo,
}: AnimatedFormProps) {
  const [visible, setVisible] = useState<boolean>(false);
  const [errors, setErrors] = useState<Errors>({});

  const toggleVisibility = () => setVisible(!visible);

  const validateForm = (event: FormEvent<HTMLFormElement>) => {
    const currentErrors: Errors = {};
    fields.forEach((field) => {
      const value = (event.target as HTMLFormElement)[field.label]?.value;

      if (field.required && !value) {
        currentErrors[field.label] = `${field.label} is required`;
      }

      if (field.type === 'email' && value && !/\S+@\S+\.\S+/.test(value)) {
        currentErrors[field.label] = 'Invalid email address';
      }
```

```
    if (field.type === 'password' && value && value.length < 6) {
      currentErrors[field.label] =
        'Password must be at least 6 characters long';
    }
  });
  return currentErrors;
};

const handleSubmit = (event: FormEvent<HTMLFormElement>) => {
  event.preventDefault();
  const formErrors = validateForm(event);

  if (Object.keys(formErrors).length === 0) {
    onSubmit(event);
    console.log('Form submitted');
  } else {
    setErrors(formErrors);
  }
};

return (
  <section className='max-md:w-full flex flex-col gap-4 w-96 mx-auto'>
    <BoxReveal boxColor='var(--skeleton)' duration={0.3}>
      <h2 className='font-bold text-3xl text-neutral-800 dark:text-neutral-200'>
        {header}
      </h2>
    </BoxReveal>
```

```
{subHeader && (
  <BoxReveal boxColor='var(--skeleton)' duration={0.3} className='pb-2'>
    <p className='text-neutral-600 text-sm max-w-sm dark:text-neutral-300'>
      {subHeader}
    </p>
  </BoxReveal>
)}

{googleLogin && (
  <>
    <BoxReveal
      boxColor='var(--skeleton)'
      duration={0.3}
      overflow='visible'
      width='unset'
    >
      <button
        className='g-button group/btn bg-transparent w-full rounded-md border h-10 font-medium outline-hidden hover:cursor-pointer'
        type='button'
        onClick={() => console.log('Google login clicked')}
      >
        <span className='flex items-center justify-center w-full h-full gap-3'>
          <Image
            src='https://cdn1.iconfinder.com/data/icons/google-s-logo/150/Google_Icons-09-512.png'
            width={26}
            height={26}
```

```
              alt='Google Icon'
             />
             {googleLogin}
            </span>

            <BottomGradient />
           </button>
          </BoxReveal>

          <BoxReveal boxColor='var(--skeleton)' duration={0.3} width='100%'>
           <section className='flex items-center gap-4'>
            <hr className='flex-1 border-1 border-dashed border-neutral-300 dark:border-neutral-700' />
            <p className='text-neutral-700 text-sm dark:text-neutral-300'>
             or
            </p>
            <hr className='flex-1 border-1 border-dashed border-neutral-300 dark:border-neutral-700' />
           </section>
          </BoxReveal>
         </>
        )}

        <form onSubmit={handleSubmit}>
         <section
          className={`grid grid-cols-1 md:grid-cols-${fieldPerRow} mb-4`}
         >
          {fields.map((field) => (
           <section key={field.label} className='flex flex-col gap-2'>
```

```
<BoxReveal boxColor='var(--skeleton)' duration={0.3}>
  <Label htmlFor={field.label}>
    {field.label} <span className='text-red-500'>*</span>
  </Label>
</BoxReveal>

<BoxReveal
  width='100%'
  boxColor='var(--skeleton)'
  duration={0.3}
  className='flex flex-col space-y-2 w-full'
>
  <section className='relative'>
    <Input
      type={
        field.type === 'password'
          ? visible
            ? 'text'
            : 'password'
          : field.type
      }
      id={field.label}
      placeholder={field.placeholder}
      onChange={field.onChange}
    />

    {field.type === 'password' && (
      <button
```

```
            type='button'
            onClick={toggleVisibility}
            className='absolute inset-y-0 right-0 pr-3 flex items-center text-sm leading-5'
          >
            {visible ? (
              <Eye className='h-5 w-5' />
            ) : (
              <EyeOff className='h-5 w-5' />
            )}
          </button>
        )}
      </section>

      <section className='h-4'>
        {errors[field.label] && (
          <p className='text-red-500 text-xs'>
            {errors[field.label]}
          </p>
        )}
      </section>
    </BoxReveal>
  </section>
 ))}
</section>

<BoxReveal width='100%' boxColor='var(--skeleton)' duration={0.3}>
 {errorField && (
  <p className='text-red-500 text-sm mb-4'>{errorField}</p>
```

```
        )}
      </BoxReveal>

      <BoxReveal
        width='100%'
        boxColor='var(--skeleton)'
        duration={0.3}
        overflow='visible'
      >
        <button
          className='bg-gradient-to-br relative group/btn from-zinc-200 dark:from-zinc-900
          dark:to-zinc-900 to-zinc-200 block dark:bg-zinc-800 w-full text-black
          dark:text-white rounded-md h-10 font-medium shadow-
[0px_1px_0px_0px_#ffffff40_inset,0px_-1px_0px_0px_#ffffff40_inset]
            dark:shadow-[0px_1px_0px_0px_var(--zinc-800)_inset,0px_-1px_0px_0px_var(--zinc-
800)_inset] outline-hidden hover:cursor-pointer'
          type='submit'
        >
          {submitButton} &rarr;
          <BottomGradient />
        </button>
      </BoxReveal>

      {textVariantButton && goTo && (
        <BoxReveal boxColor='var(--skeleton)' duration={0.3}>
          <section className='mt-4 text-center hover:cursor-pointer'>
            <button
              className='text-sm text-blue-500 hover:cursor-pointer outline-hidden'
              onClick={goTo}
```

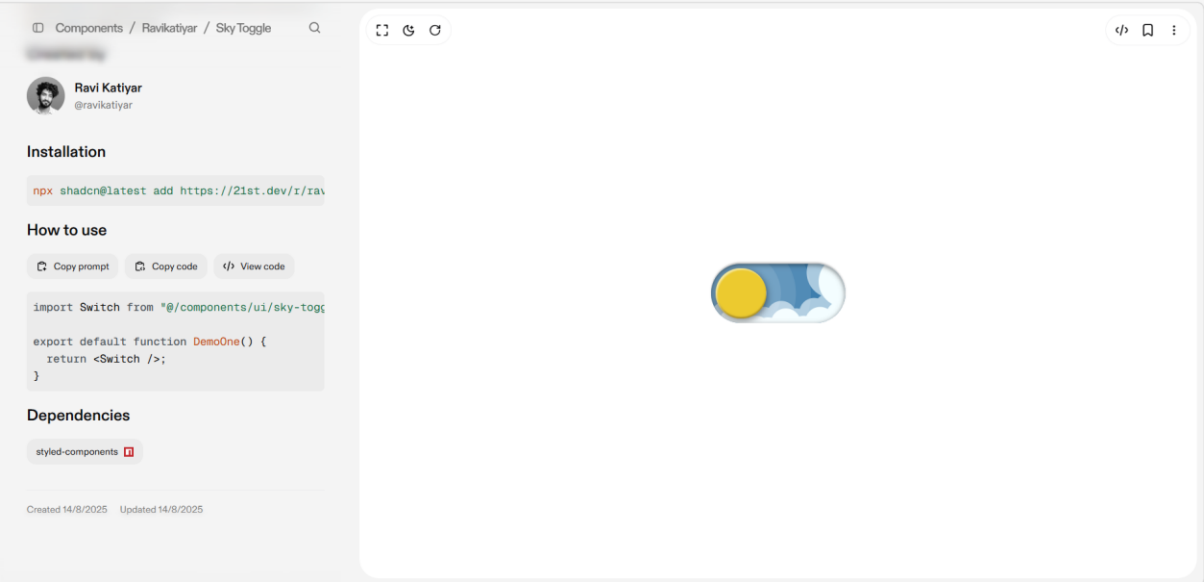
```
            >
              {textVariantButton}
            </button>
          </section>
        </BoxReveal>
      )}
    </form>
  </section>
 );
});

const BottomGradient = () => {
 return (
  <>
   <span className='group-hover/btn:opacity-100 block transition duration-500 opacity-0 absolute h-px w-full -bottom-px inset-x-0 bg-gradient-to-r from-transparent via-cyan-500 to-transparent' />
   <span className='group-hover/btn:opacity-100 blur-sm block transition duration-500 opacity-0 absolute h-px w-1/2 mx-auto -bottom-px inset-x-10 bg-gradient-to-r from-transparent via-indigo-500 to-transparent' />
  </>
 );
};

// ==================== AuthTabs Component ====================

interface AuthTabsProps {
 formFields: {
  header: string;
```

```
    subHeader?: string;
    fields: Array<{
      label: string;
      required?: boolean;
      type: string;
      placeholder: string;
      onChange: (event: React.ChangeEvent<HTMLInputElement>) => void;
    }>;
    submitButton: string;
    textVariantButton?: string;
  };
  goTo: (event: React.MouseEvent<HTMLButtonElement>) => void;
  handleSubmit: (event: React.FormEvent<HTMLFormElement>) => void;
}

const AuthTabs = memo(function AuthTabs({
  formFields,
  goTo,
  handleSubmit,
}: AuthTabsProps) {
  return (
    <div className='flex max-lg:justify-center w-full md:w-auto'>
      {/* Right Side */}
      <div className='w-full lg:w-1/2 h-[100dvh] flex flex-col justify-center items-center max-lg:px-[10%]'>
        <AnimatedForm
          {...formFields}
          fieldPerRow={1}
```

```
        onSubmit={handleSubmit}
        goTo={goTo}
        googleLogin='Login with Google'
       />
     </div>
   </div>
  );
});

// ==================== Label Component ====================

interface LabelProps extends React.LabelHTMLAttributes<HTMLLabelElement> {
  htmlFor?: string;
}

const Label = memo(function Label({ className, ...props }: LabelProps) {
  return (
    <label
      className={cn(
        'text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70',
        className
      )}
      {...props}
    />
  );
});
```

```
// ==================== Exports ====================

export {
  Input,
  BoxReveal,
  Ripple,
  OrbitingCircles,
  TechOrbitDisplay,
  AnimatedForm,
  AuthTabs,
  Label,
  BottomGradient,
};
```

5.For toggle for dark and light mode:



ch

```
import React from 'react';
import styled from 'styled-components';


const Switch = () => {
  return (
    <StyledWrapper>
      <label className="theme-switch">
        <input type="checkbox" className="theme-switch__checkbox" />
        <div className="theme-switch__container">
          <div className="theme-switch__clouds" />
          <div className="theme-switch__stars-container">
            <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 144 55" fill="none">
              <path fillRule="evenodd" clipRule="evenodd" d="M135.831 3.00688C135.055
3.85027 134.111 4.29946 133 4.35447C134.111 4.40947 135.055 4.85867 135.831
5.71123C136.607 6.55462 136.996 7.56303 136.996 8.72727C136.996 7.95722 137.172
7.25134 137.525 6.59129C137.886 5.93124 138.372 5.39954 138.98 5.00535C139.598
4.60199 140.268 4.39114 141 4.35447C139.88 4.2903 138.936 3.85027 138.16
3.00688C137.384 2.16348 136.996 1.16425 136.996 0C136.996 1.16425 136.607 2.16348
135.831 3.00688ZM31 23.3545C32.1114 23.2995 33.0551 22.8503 33.8313
22.0069C34.6075 21.1635 34.9956 20.1642 34.9956 19C34.9956 20.1642 35.3837 21.1635
36.1599 22.0069C36.9361 22.8503 37.8798 23.2903 39 23.3545C38.2679 23.3911 37.5976
```

23.602 36.9802 24.0053C36.3716 24.3995 35.8864 24.9312 35.5248 25.5913C35.172
26.2513 34.9956 26.9572 34.9956 27.7273C34.9956 26.563 34.6075 25.5546 33.8313
24.7112C33.0551 23.8587 32.1114 23.4095 31 23.3545ZM0 36.3545C1.11136 36.2995
2.05513 35.8503 2.83131 35.0069C3.6075 34.1635 3.99559 33.1642 3.99559 32C3.99559
33.1642 4.38368 34.1635 5.15987 35.0069C5.93605 35.8503 6.87982 36.2903 8
36.3545C7.26792 36.3911 6.59757 36.602 5.98015 37.0053C5.37155 37.3995 4.88644
37.9312 4.52481 38.5913C4.172 39.2513 3.99559 39.9572 3.99559 40.7273C3.99559 39.563
3.6075 38.5546 2.83131 37.7112C2.05513 36.8587 1.11136 36.4095 0 36.3545ZM56.8313
24.0069C56.0551 24.8503 55.1114 25.2995 54 25.3545C55.1114 25.4095 56.0551 25.8587
56.8313 26.7112C57.6075 27.5546 57.9956 28.563 57.9956 29.7273C57.9956 28.9572
58.172 28.2513 58.5248 27.5913C58.8864 26.9312 59.3716 26.3995 59.9802
26.0053C60.5976 25.602 61.2679 25.3911 62 25.3545C60.8798 25.2903 59.9361 24.8503
59.1599 24.0069C58.3837 23.1635 57.9956 22.1642 57.9956 21C57.9956 22.1642 57.6075
23.1635 56.8313 24.0069ZM81 25.3545C82.1114 25.2995 83.0551 24.8503 83.8313
24.0069C84.6075 23.1635 84.9956 22.1642 84.9956 21C84.9956 22.1642 85.3837 23.1635
86.1599 24.0069C86.9361 24.8503 87.8798 25.2903 89 25.3545C88.2679 25.3911 87.5976
25.602 86.9802 26.0053C86.3716 26.3995 85.8864 26.9312 85.5248 27.5913C85.172
28.2513 84.9956 28.9572 84.9956 29.7273C84.9956 28.563 84.6075 27.5546 83.8313
26.7112C83.0551 25.8587 82.1114 25.4095 81 25.3545ZM136 36.3545C137.111 36.2995
138.055 35.8503 138.831 35.0069C139.607 34.1635 139.996 33.1642 139.996 32C139.996
33.1642 140.384 34.1635 141.16 35.0069C141.936 35.8503 142.88 36.2903 144
36.3545C143.268 36.3911 142.598 36.602 141.98 37.0053C141.372 37.3995 140.886
37.9312 140.525 38.5913C140.172 39.2513 139.996 39.9572 139.996 40.7273C139.996
39.563 139.607 38.5546 138.831 37.7112C138.055 36.8587 137.111 36.4095 136
36.3545ZM101.831 49.0069C101.055 49.8503 100.111 50.2995 99 50.3545C100.111
50.4095 101.055 50.8587 101.831 51.7112C102.607 52.5546 102.996 53.563 102.996
54.7273C102.996 53.9572 103.172 53.2513 103.525 52.5913C103.886 51.9312 104.372
51.3995 104.98 51.0053C105.598 50.602 106.268 50.3911 107 50.3545C105.88 50.2903
104.936 49.8503 104.16 49.0069C103.384 48.1635 102.996 47.1642 102.996 46C102.996
47.1642 102.607 48.1635 101.831 49.0069Z" fill="currentColor" />

</svg>

```
            <div className="theme-switch__spot" />
          </div>
        </div>
      </div>
    </label>
  </StyledWrapper>
 );
}

const StyledWrapper = styled.div`
  .theme-switch {
    --toggle-size: 30px;
    /* the size is adjusted using font-size,
       this is not transform scale,
       so you can choose any size */
    --container-width: 5.625em;
    --container-height: 2.5em;
    --container-radius: 6.25em;
    /* radius 0 - minecraft mode :) */
    --container-light-bg: #3D7EAE;
    --container-night-bg: #1D1F2C;
    --circle-container-diameter: 3.375em;
    --sun-moon-diameter: 2.125em;
    --sun-bg: #ECCA2F;
    --moon-bg: #C4C9D1;
    --spot-color: #959DB1;
```

```
  --circle-container-offset: calc((var(--circle-container-diameter) - var(--container-height)) / 2 * -1);
  --stars-color: #fff;
  --clouds-color: #F3FDFF;
  --back-clouds-color: #AACADF;
  --transition: .5s cubic-bezier(0, -0.02, 0.4, 1.25);
  --circle-transition: .3s cubic-bezier(0, -0.02, 0.35, 1.17);
 }

 .theme-switch, .theme-switch *, .theme-switch *::before, .theme-switch *::after {
  -webkit-box-sizing: border-box;
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-size: var(--toggle-size);
 }

 .theme-switch__container {
  width: var(--container-width);
  height: var(--container-height);
  background-color: var(--container-light-bg);
  border-radius: var(--container-radius);
  overflow: hidden;
  cursor: pointer;
  -webkit-box-shadow: 0em -0.062em 0.062em rgba(0, 0, 0, 0.25), 0em 0.062em 0.125em rgba(255, 255, 255, 0.94);
  box-shadow: 0em -0.062em 0.062em rgba(0, 0, 0, 0.25), 0em 0.062em 0.125em rgba(255, 255, 255, 0.94);
  -webkit-transition: var(--transition);
```

```
  -o-transition: var(--transition);
  transition: var(--transition);
  position: relative;
 }

 .theme-switch__container::before {
  content: "";
  position: absolute;
  z-index: 1;
  inset: 0;
  -webkit-box-shadow: 0em 0.05em 0.187em rgba(0, 0, 0, 0.25) inset, 0em 0.05em 0.187em rgba(0, 0, 0, 0.25) inset;
  box-shadow: 0em 0.05em 0.187em rgba(0, 0, 0, 0.25) inset, 0em 0.05em 0.187em rgba(0, 0, 0, 0.25) inset;
  border-radius: var(--container-radius)
 }

 .theme-switch__checkbox {
  display: none;
 }

 .theme-switch__circle-container {
  width: var(--circle-container-diameter);
  height: var(--circle-container-diameter);
  background-color: rgba(255, 255, 255, 0.1);
  position: absolute;
  left: var(--circle-container-offset);
  top: var(--circle-container-offset);
  border-radius: var(--container-radius);
```

```
    -webkit-box-shadow: inset 0 0 0 3.375em rgba(255, 255, 255, 0.1), inset 0 0 0 3.375em rgba(255, 255, 255, 0.1), 0 0 0 0.625em rgba(255, 255, 255, 0.1), 0 0 0 1.25em rgba(255, 255, 255, 0.1);
    box-shadow: inset 0 0 0 3.375em rgba(255, 255, 255, 0.1), inset 0 0 0 3.375em rgba(255, 255, 255, 0.1), 0 0 0 0.625em rgba(255, 255, 255, 0.1), 0 0 0 1.25em rgba(255, 255, 255, 0.1);
    display: -webkit-box;
    display: -ms-flexbox;
    display: flex;
    -webkit-transition: var(--circle-transition);
    -o-transition: var(--circle-transition);
    transition: var(--circle-transition);
    pointer-events: none;
  }

  .theme-switch__sun-moon-container {
    pointer-events: auto;
    position: relative;
    z-index: 2;
    width: var(--sun-moon-diameter);
    height: var(--sun-moon-diameter);
    margin: auto;
    border-radius: var(--container-radius);
    background-color: var(--sun-bg);
    -webkit-box-shadow: 0.062em 0.062em 0.062em 0em rgba(254, 255, 239, 0.61) inset, 0em -0.062em 0.062em 0em #a1872a inset;
    box-shadow: 0.062em 0.062em 0.062em 0em rgba(254, 255, 239, 0.61) inset, 0em -0.062em 0.062em 0em #a1872a inset;
    -webkit-filter: drop-shadow(0.062em 0.125em 0.125em rgba(0, 0, 0, 0.25)) drop-shadow(0em 0.062em 0.125em rgba(0, 0, 0, 0.25));
```

```
    filter: drop-shadow(0.062em 0.125em 0.125em rgba(0, 0, 0, 0.25)) drop-shadow(0em
0.062em 0.125em rgba(0, 0, 0, 0.25));
    overflow: hidden;
    -webkit-transition: var(--transition);
    -o-transition: var(--transition);
    transition: var(--transition);
  }

  .theme-switch__moon {
    -webkit-transform: translateX(100%);
    -ms-transform: translateX(100%);
    transform: translateX(100%);
    width: 100%;
    height: 100%;
    background-color: var(--moon-bg);
    border-radius: inherit;
    -webkit-box-shadow: 0.062em 0.062em 0.062em 0em rgba(254, 255, 239, 0.61) inset,
0em -0.062em 0.062em 0em #969696 inset;
    box-shadow: 0.062em 0.062em 0.062em 0em rgba(254, 255, 239, 0.61) inset, 0em -
0.062em 0.062em 0em #969696 inset;
    -webkit-transition: var(--transition);
    -o-transition: var(--transition);
    transition: var(--transition);
    position: relative;
  }

  .theme-switch__spot {
    position: absolute;
    top: 0.75em;
```

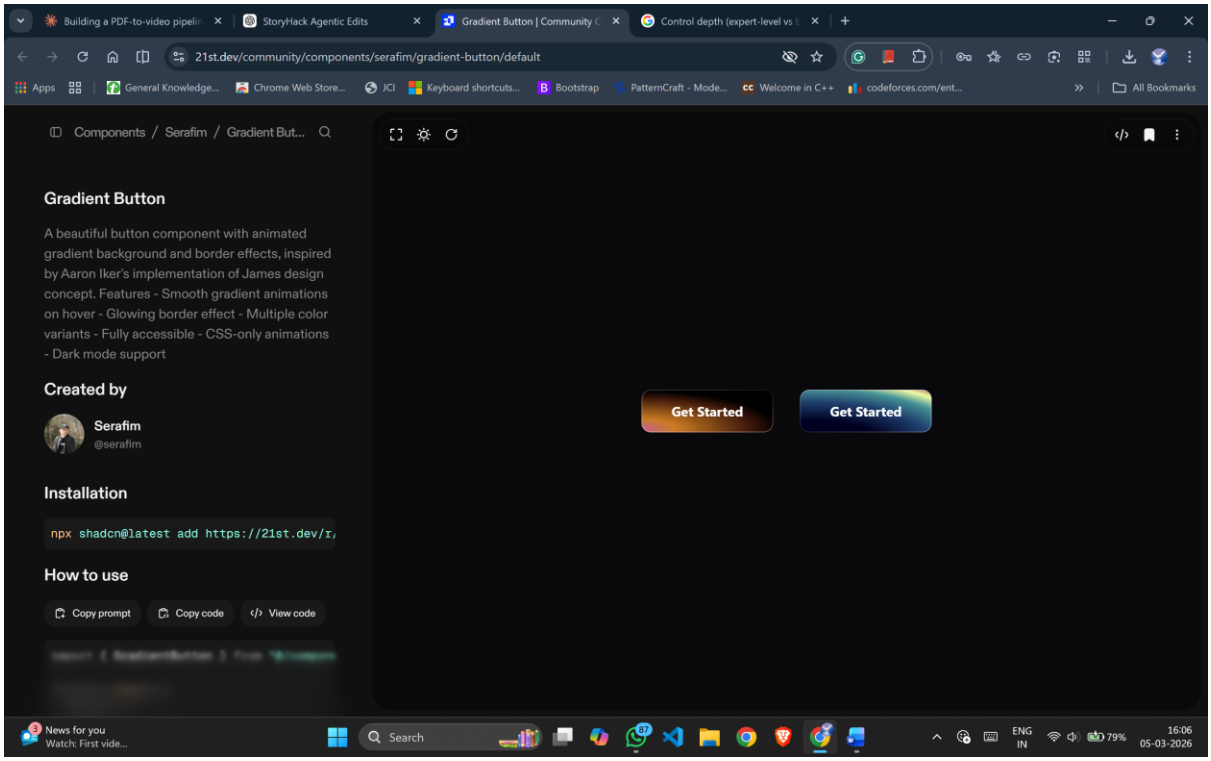
```
  left: 0.312em;
  width: 0.75em;
  height: 0.75em;
  border-radius: var(--container-radius);
  background-color: var(--spot-color);
  -webkit-box-shadow: 0em 0.0312em 0.062em rgba(0, 0, 0, 0.25) inset;
  box-shadow: 0em 0.0312em 0.062em rgba(0, 0, 0, 0.25) inset;
}

.theme-switch__spot:nth-of-type(2) {
  width: 0.375em;
  height: 0.375em;
  top: 0.937em;
  left: 1.375em;
}

.theme-switch__spot:nth-last-of-type(3) {
  width: 0.25em;
  height: 0.25em;
  top: 0.312em;
  left: 0.812em;
}

.theme-switch__clouds {
  width: 1.25em;
  height: 1.25em;
  background-color: var(--clouds-color);
  border-radius: var(--container-radius);
```

```
  position: absolute;
  bottom: -0.625em;
  left: 0.312em;
  -webkit-box-shadow: 0.937em 0.312em var(--clouds-color), -0.312em -0.312em var(--back-clouds-color), 1.437em 0.375em var(--clouds-color), 0.5em -0.125em var(--back-clouds-color), 2.187em 0 var(--clouds-color), 1.25em -0.062em var(--back-clouds-color), 2.937em 0.312em var(--clouds-color), 2em -0.312em var(--back-clouds-color), 3.625em -0.062em var(--clouds-color), 2.625em 0em var(--back-clouds-color), 4.5em -0.312em var(--clouds-color), 3.375em -0.437em var(--back-clouds-color), 4.625em -1.75em 0 0.437em var(--clouds-color), 4em -0.625em var(--back-clouds-color), 4.125em -2.125em 0 0.437em var(--back-clouds-color);
  box-shadow: 0.937em 0.312em var(--clouds-color), -0.312em -0.312em var(--back-clouds-color), 1.437em 0.375em var(--clouds-color), 0.5em -0.125em var(--back-clouds-color), 2.187em 0 var(--clouds-color), 1.25em -0.062em var(--back-clouds-color), 2.937em 0.312em var(--clouds-color), 2em -0.312em var(--back-clouds-color), 3.625em -0.062em var(--clouds-color), 2.625em 0em var(--back-clouds-color), 4.5em -0.312em var(--clouds-color), 3.375em -0.437em var(--back-clouds-color), 4.625em -1.75em 0 0.437em var(--clouds-color), 4em -0.625em var(--back-clouds-color), 4.125em -2.125em 0 0.437em var(--back-clouds-color);
  -webkit-transition: 0.5s cubic-bezier(0, -0.02, 0.4, 1.25);
  -o-transition: 0.5s cubic-bezier(0, -0.02, 0.4, 1.25);
  transition: 0.5s cubic-bezier(0, -0.02, 0.4, 1.25);
 }

 .theme-switch__stars-container {
  position: absolute;
  color: var(--stars-color);
  top: -100%;
  left: 0.312em;
  width: 2.75em;
  height: auto;
  -webkit-transition: var(--transition);
  -o-transition: var(--transition);
```

```
  transition: var(--transition);
 }

 /* actions */

 .theme-switch__checkbox:checked + .theme-switch__container {
  background-color: var(--container-night-bg);
 }

 .theme-switch__checkbox:checked + .theme-switch__container .theme-switch__circle-container {
  left: calc(100% - var(--circle-container-offset) - var(--circle-container-diameter));
 }

 .theme-switch__checkbox:checked + .theme-switch__container .theme-switch__circle-container:hover {
  left: calc(100% - var(--circle-container-offset) - var(--circle-container-diameter) - 0.187em)
 }

 .theme-switch__circle-container:hover {
  left: calc(var(--circle-container-offset) + 0.187em);
 }

 .theme-switch__checkbox:checked + .theme-switch__container .theme-switch__moon {
  -webkit-transform: translate(0);
  -ms-transform: translate(0);
  transform: translate(0);
 }
```

```
 .theme-switch__checkbox:checked + .theme-switch__container .theme-switch__clouds {
   bottom: -4.062em;
 }

 .theme-switch__checkbox:checked + .theme-switch__container .theme-switch__stars-container {
   top: 50%;
   -webkit-transform: translateY(-50%);
   -ms-transform: translateY(-50%);
   transform: translateY(-50%);
 }`;

export default Switch;
```

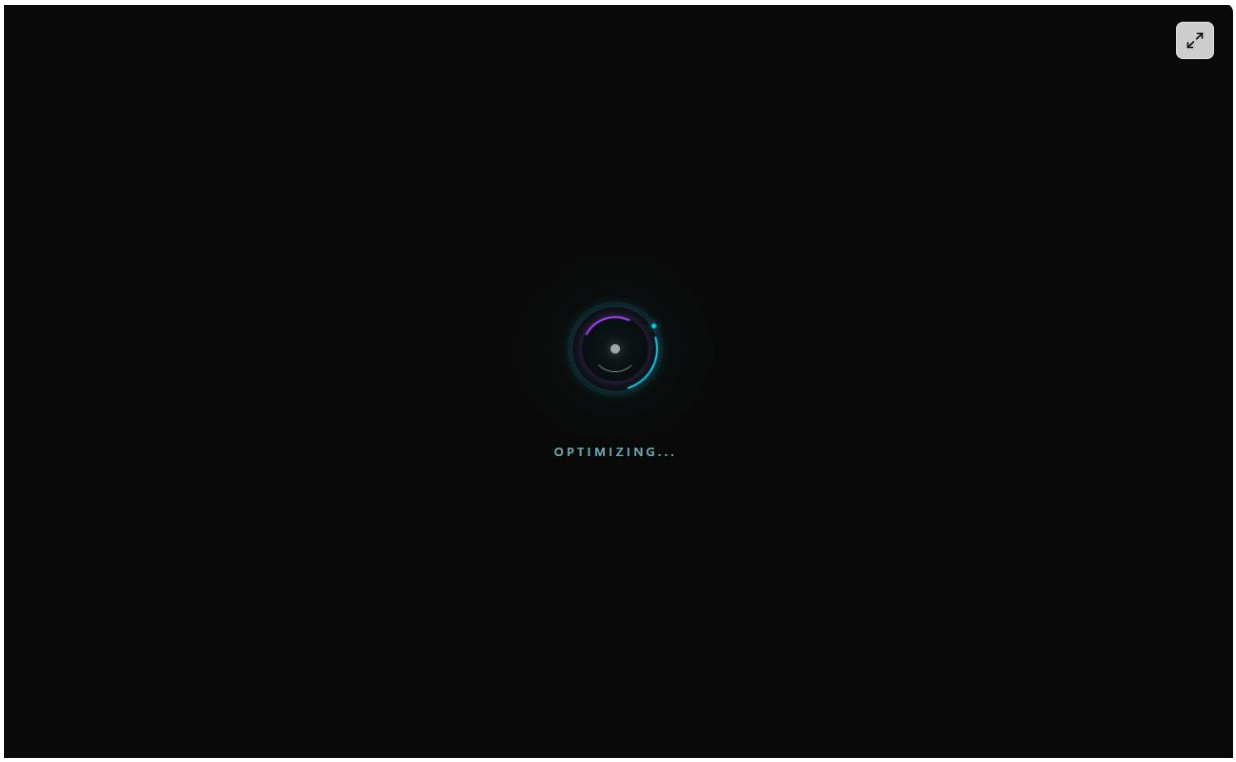
6.For buttons:



```
"use client"

import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva, type VariantProps } from "class-variance-authority"
import { cn } from "@/lib/utils"

const gradientButtonVariants = cva(
  [
    "gradient-button",
    "inline-flex items-center justify-center",
    "rounded-[11px] min-w-[132px] px-9 py-4",
    "text-base leading-[19px] font-[500] text-white",
    "font-sans font-bold",
    "focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring",
    "disabled:pointer-events-none disabled:opacity-50",
  ],
  {
    variants: {
      variant: {
        default: "",
        variant: "gradient-button-variant",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
```

```
)

export interface GradientButtonProps
  extends React.ButtonHTMLAttributes<HTMLButtonElement>,
    VariantProps<typeof gradientButtonVariants> {
  asChild?: boolean
}

const GradientButton = React.forwardRef<HTMLButtonElement, GradientButtonProps>(
  ({ className, variant, asChild = false, ...props }, ref) => {
    const Comp = asChild ? Slot : "button"
    return (
      <Comp
        className={cn(gradientButtonVariants({ variant, className }))}
        ref={ref}
        {...props}
      />
    )
  }
)
GradientButton.displayName = "GradientButton"

export { GradientButton, gradientButtonVariants }
```

7.For loading and while generating the results use this:




```
'use client'

import React, { useState, useEffect } from 'react'

export function CoreSpinLoader() {
  const [loadingText, setLoadingText] = useState('Initializing')

  useEffect(() => {
    const states = ['Loading...', 'Fetching Data..', 'Syncing...', 'Processing..', 'Optimizing...']
    let i = 0
    const interval = setInterval(() => {
      i = (i + 1) % states.length
      setLoadingText(states[i])
    }, 1000)
```

```
  return () => clearInterval(interval)
}, [])

return (
  <div className="flex flex-col items-center justify-center min-h-[200px] gap-8">
    <div className="relative w-20 h-20 flex items-center justify-center">

      {/* Base Glow */}
      <div className="
        absolute inset-0 rounded-full blur-xl animate-pulse
        bg-emerald-400/15
        dark:bg-cyan-500/10
      " />

      {/* Outer Dashed Ring */}
      <div className="
        absolute inset-0 rounded-full border border-dashed
        border-emerald-500/40
        dark:border-cyan-500/20
        animate-[spin_10s_linear_infinite]
      " />

      {/* Main Arc */}
      <div className="
        absolute inset-1 rounded-full border-2 border-transparent
        border-t-emerald-500
        dark:border-t-cyan-400
        shadow-[0_0_6px_rgba(16,185,129,0.5)]
```

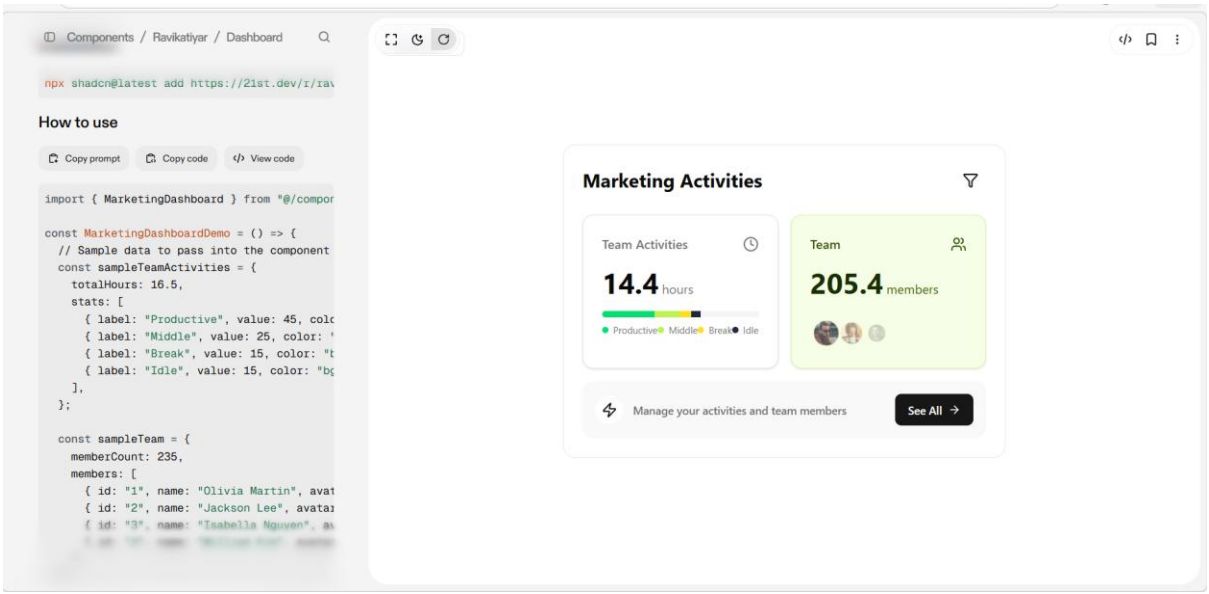
```
  dark:shadow-[0_0_10px_rgba(34,211,238,0.4)]
  animate-[spin_2s_linear_infinite]
" />

{/* Reverse Arc */}
<div className="
  absolute inset-3 rounded-full border-2 border-transparent
  border-b-green-600
  dark:border-b-purple-500
  shadow-[0_0_6px_rgba(22,163,74,0.4)]
  dark:shadow-[0_0_10px_rgba(168,85,247,0.4)]
  animate-[spin_3s_linear_infinite_reverse]
" />

{/* Inner Fast Ring */}
<div className="
  absolute inset-5 rounded-full border border-transparent
  border-l-green-700/60
  dark:border-l-white/50
  animate-[spin_1s_ease-in-out_infinite]
" />

{/* Orbital Dot */}
<div className="absolute inset-0 animate-[spin_4s_linear_infinite]">
  <div className="
    absolute top-0 left-1/2 -translate-x-1/2
    w-1 h-1 rounded-full
    bg-emerald-600
```

```
    dark:bg-cyan-400
    shadow-[0_0_4px_rgba(16,185,129,0.9)]
    dark:shadow-[0_0_6px_rgba(34,211,238,0.8)]
   " />
  </div>

  {/* Center Core */}
  <div className="
   absolute w-2 h-2 rounded-full animate-pulse
   bg-emerald-700
   dark:bg-white
   shadow-[0_0_6px_rgba(16,185,129,0.6)]
   dark:shadow-[0_0_10px_rgba(255,255,255,0.8)]
  " />
 </div>

 {/* Text */}
 <div className="flex flex-col items-center gap-1 h-8 justify-center">
  <span
   key={loadingText}
   className="
    text-[10px] font-medium tracking-[0.3em] uppercase
    text-emerald-700
    dark:text-cyan-200/70
    animate-in fade-in slide-in-from-bottom-2 duration-500
   "
  >
   {loadingText}
```

```
      </span>
    </div>
  </div>
 )
}
```

8.

If needed, the count should be shown like this



s

```
import * as React from "react";
import { motion, useMotionValue, useTransform, animate } from "framer-motion";
import { Filter, Users, Clock, Zap, ArrowRight } from "lucide-react";
import { cn } from "@/lib/utils"; // Your utility for merging class names
import { Button } from "@/components/ui/button"; // Assuming you have a Button component
import { Card, CardContent } from "@/components/ui/card";
import { Avatar, AvatarFallback, AvatarImage } from "@/components/ui/avatar";
```

```
// Type definitions for component props
interface ActivityStat {
  label: string;
  value: number; // Represents percentage
  color: string; // Tailwind color class e.g., 'bg-green-400'
}

interface TeamMember {
  id: string;
  name: string;
  avatarUrl: string;
}

interface MarketingDashboardProps {
  title?: string;
  teamActivities: {
    totalHours: number;
    stats: ActivityStat[];
  };
  team: {
    memberCount: number;
    members: TeamMember[];
  };
  cta: {
    text: string;
    buttonText: string;
    onButtonClick: () => void;
```

```
  };
  onFilterClick?: () => void;
  className?: string;
}

// Sub-component for animating numbers
const AnimatedNumber = ({ value }: { value: number }) => {
  const count = useMotionValue(0);
  const rounded = useTransform(count, (latest) => Math.round(latest * 10) / 10); // Format to one decimal place

  React.useEffect(() => {
    const controls = animate(count, value, {
      duration: 1.5,
      ease: "easeOut",
    });
    return controls.stop;
  }, [value, count]);

  return <motion.span>{rounded}</motion.span>;
};

// Main Component
export const MarketingDashboard = React.forwardRef<
  HTMLDivElement,
  MarketingDashboardProps
>(({
  title = "Marketing Activities",
```

```
  teamActivities,
  team,
  cta,
  onFilterClick,
  className
}, ref) => {

  // Animation variants for Framer Motion
  const containerVariants = {
    hidden: { opacity: 0, y: 20 },
    visible: {
      opacity: 1,
      y: 0,
      transition: {
        staggerChildren: 0.1,
      },
    },
  };

  const itemVariants = {
    hidden: { opacity: 0, y: 15 },
    visible: { opacity: 1, y: 0, transition: { duration: 0.5 } },
  };

  const hoverTransition = { type: "spring", stiffness: 300, damping: 15 };

  return (
    <motion.div
```

```
    ref={ref}
    className={cn("w-full max-w-2xl p-6 bg-card text-card-foreground rounded-2xl border", className)}
    variants={containerVariants}
    initial="hidden"
    animate="visible"
   >
    {/* Header */}
    <motion.div variants={itemVariants} className="flex items-center justify-between mb-6">
     <h2 className="text-2xl font-bold">{title}</h2>
     <Button variant="ghost" size="icon" onClick={onFilterClick} aria-label="Filter activities">
      <Filter className="w-5 h-5" />
     </Button>
    </motion.div>

    {/* Stats Grid */}
    <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
     {/* Team Activities Card */}
     <motion.div
      variants={itemVariants}
      whileHover={{ scale: 1.03, y: -5 }} // Added for hover effect
      transition={hoverTransition} // Added for hover effect
     >
      <Card className="h-full p-4 overflow-hidden rounded-xl">
       <CardContent className="p-2">
        <div className="flex items-center justify-between mb-4">
         <p className="font-medium text-muted-foreground">Team Activities</p>
         <Clock className="w-5 h-5 text-muted-foreground" />
```

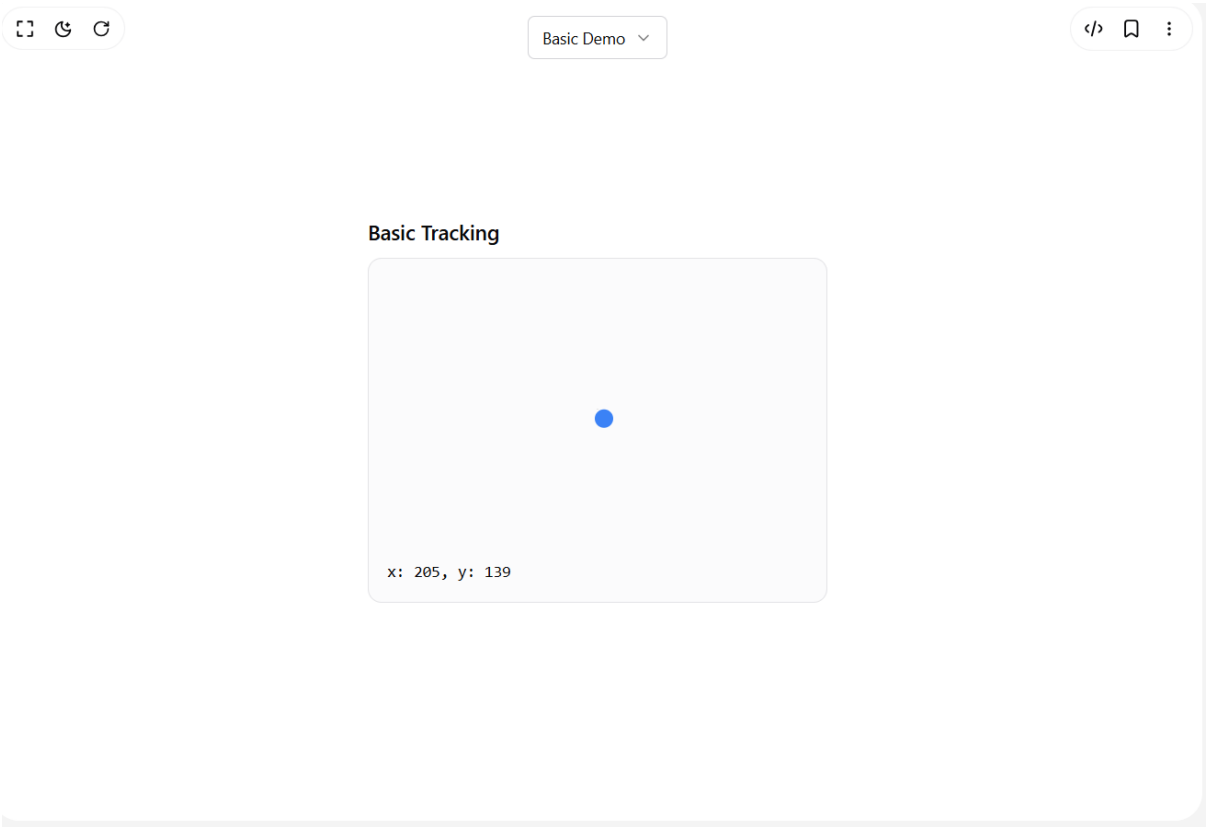
```
</div>
<div className="mb-4">
  <span className="text-4xl font-bold">
    <AnimatedNumber value={teamActivities.totalHours} />
  </span>
  <span className="ml-1 text-muted-foreground">hours</span>
</div>
{/* Progress Bar */}
<div className="w-full h-2 mb-2 overflow-hidden rounded-full bg-muted flex">
  {teamActivities.stats.map((stat, index) => (
    <motion.div
      key={index}
      className={cn("h-full", stat.color)}
      initial={{ width: 0 }}
      animate={{ width: `${stat.value}%` }}
      transition={{ duration: 1, delay: 0.5 + index * 0.1 }}
    />
  ))}
</div>
{/* Legend */}
<div className="flex items-center justify-between text-xs text-muted-foreground">
  {teamActivities.stats.map((stat) => (
    <div key={stat.label} className="flex items-center gap-1.5">
      <span className={cn("w-2 h-2 rounded-full", stat.color)}></span>
      <span>{stat.label}</span>
    </div>
  ))}
</div>
```

```
    </CardContent>
   </Card>
  </motion.div>

  {/* Team Members Card */}
  <motion.div
   variants={itemVariants}
   whileHover={{ scale: 1.03, y: -5 }} // Added for hover effect
   transition={hoverTransition} // Added for hover effect
  >
   <Card className="h-full p-4 overflow-hidden rounded-xl bg-lime-50 dark:bg-lime-900/30 border-lime-200 dark:border-lime-800">
    <CardContent className="p-2">
     <div className="flex items-center justify-between mb-4">
      <p className="font-medium text-lime-900 dark:text-lime-200">Team</p>
      <Users className="w-5 h-5 text-lime-900 dark:text-lime-200" />
     </div>
     <div className="mb-6">
      <span className="text-4xl font-bold text-lime-950 dark:text-lime-50">
       <AnimatedNumber value={team.memberCount} />
      </span>
      <span className="ml-1 text-lime-800 dark:text-lime-300">members</span>
     </div>
     {/* Avatar Stack */}
     <div className="flex -space-x-2">
      {team.members.slice(0, 4).map((member, index) => (
       <motion.div
        key={member.id}
```

```
            initial={{ opacity: 0, scale: 0.5 }}
            animate={{ opacity: 1, scale: 1 }}
            transition={{ duration: 0.5, delay: 0.8 + index * 0.1 }}
            whileHover={{ scale: 1.2, zIndex: 10, y: -2 }} // Added for hover effect
          >
            <Avatar className="border-2 border-lime-100 dark:border-lime-900">
              <AvatarImage src={member.avatarUrl} alt={member.name} />
              <AvatarFallback>{member.name.charAt(0)}</AvatarFallback>
            </Avatar>
          </motion.div>
        ))}
      </div>
    </CardContent>
  </Card>
 </motion.div>
</div>

{/* CTA Banner */}
<motion.div
 variants={itemVariants}
 whileHover={{ scale: 1.02 }} // Added for hover effect
 transition={hoverTransition} // Added for hover effect
 className="mt-4"
>
  <div className="flex items-center justify-between p-4 rounded-xl bg-muted/60">
    <div className="flex items-center gap-3">
      <div className="p-2 rounded-full bg-background">
        <Zap className="w-5 h-5 text-foreground" />
```

```
            </div>
            <p className="text-sm font-medium text-muted-foreground">{cta.text}</p>
          </div>
          <Button onClick={cta.onButtonClick} className="shrink-0">
            {cta.buttonText}
            <ArrowRight className="w-4 h-4 ml-2" />
          </Button>
        </div>
      </motion.div>
    </motion.div>
  );
});
MarketingDashboard.displayName = "MarketingDashboard";
```

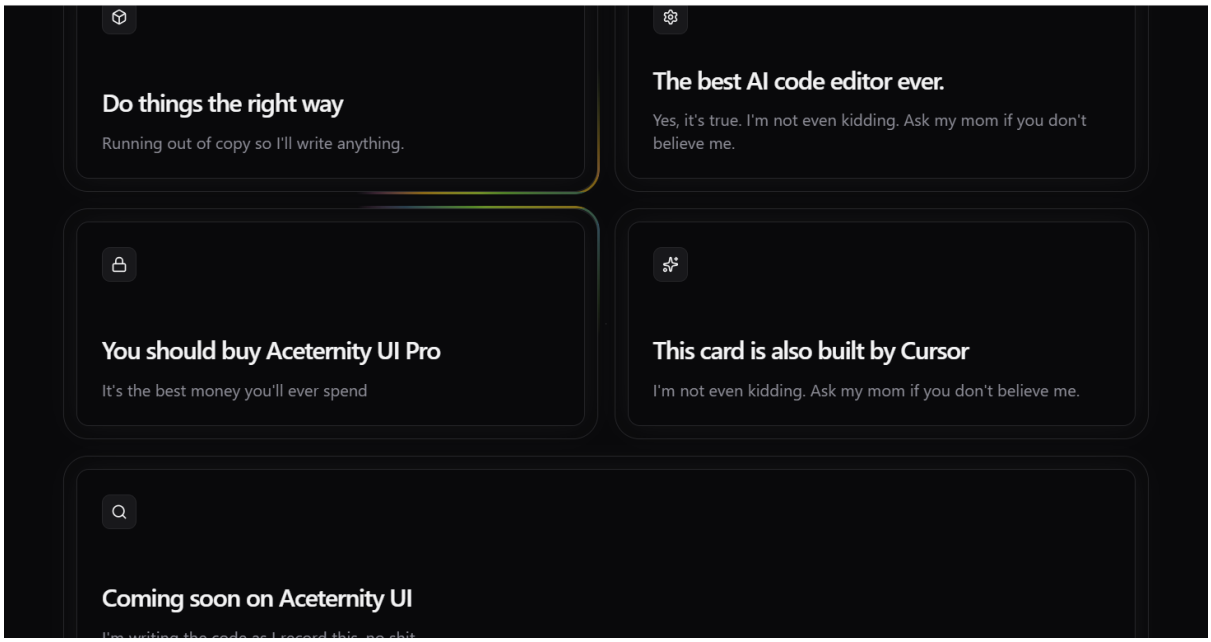
9.For gradient mouse cursor:



```
import { RefObject, useEffect, useRef } from "react";

export const useMousePositionRef = (
  containerRef?: RefObject<HTMLElement | SVGElement>
) => {
  const positionRef = useRef({ x: 0, y: 0 });

  useEffect(() => {
    const updatePosition = (x: number, y: number) => {
      if (containerRef && containerRef.current) {
        const rect = containerRef.current.getBoundingClientRect();
        const relativeX = x - rect.left;
        const relativeY = y - rect.top;

        // Calculate relative position even when outside the container
        positionRef.current = { x: relativeX, y: relativeY };
      } else {
        positionRef.current = { x, y };
      }
    };

    const handleMouseMove = (ev: MouseEvent) => {
      updatePosition(ev.clientX, ev.clientY);
    };

    const handleTouchMove = (ev: TouchEvent) => {
      const touch = ev.touches[0];
      updatePosition(touch.clientX, touch.clientY);
```

```
  };

  // Listen for both mouse and touch events
  window.addEventListener("mousemove", handleMouseMove);
  window.addEventListener("touchmove", handleTouchMove);

  return () => {
    window.removeEventListener("mousemove", handleMouseMove);
    window.removeEventListener("touchmove", handleTouchMove);
  };
 }, [containerRef]);

 return positionRef;
};
```

10. For Container:



Code :

```
"use client";
```

```tsx
import { memo, useCallback, useEffect, useRef } from "react";
import { cn } from "@/lib/utils";
import { animate } from "motion/react";

interface GlowingEffectProps {
  blur?: number;
  inactiveZone?: number;
  proximity?: number;
  spread?: number;
  variant?: "default" | "white";
  glow?: boolean;
  className?: string;
  disabled?: boolean;
  movementDuration?: number;
  borderWidth?: number;
}
const GlowingEffect = memo(
  ({
    blur = 0,
    inactiveZone = 0.7,
    proximity = 0,
    spread = 20,
    variant = "default",
    glow = false,
    className,
    movementDuration = 2,
    borderWidth = 1,
```

```
  disabled = true,
}: GlowingEffectProps) => {
  const containerRef = useRef<HTMLDivElement>(null);
  const lastPosition = useRef({ x: 0, y: 0 });
  const animationFrameRef = useRef<number>(0);

  const handleMove = useCallback(
    (e?: MouseEvent | { x: number; y: number }) => {
      if (!containerRef.current) return;

      if (animationFrameRef.current) {
        cancelAnimationFrame(animationFrameRef.current);
      }

      animationFrameRef.current = requestAnimationFrame(() => {
        const element = containerRef.current;
        if (!element) return;

        const { left, top, width, height } = element.getBoundingClientRect();
        const mouseX = e?.x ?? lastPosition.current.x;
        const mouseY = e?.y ?? lastPosition.current.y;

        if (e) {
          lastPosition.current = { x: mouseX, y: mouseY };
        }

        const center = [left + width * 0.5, top + height * 0.5];
        const distanceFromCenter = Math.hypot(
```

```
  mouseX - center[0],
  mouseY - center[1]
);
const inactiveRadius = 0.5 * Math.min(width, height) * inactiveZone;

if (distanceFromCenter < inactiveRadius) {
  element.style.setProperty("--active", "0");
  return;
}

const isActive =
  mouseX > left - proximity &&
  mouseX < left + width + proximity &&
  mouseY > top - proximity &&
  mouseY < top + height + proximity;

element.style.setProperty("--active", isActive ? "1" : "0");

if (!isActive) return;

const currentAngle =
  parseFloat(element.style.getPropertyValue("--start")) || 0;
let targetAngle =
  (180 * Math.atan2(mouseY - center[1], mouseX - center[0])) /
    Math.PI +
  90;

const angleDiff = ((targetAngle - currentAngle + 180) % 360) - 180;
```

```
      const newAngle = currentAngle + angleDiff;

      animate(currentAngle, newAngle, {
        duration: movementDuration,
        ease: [0.16, 1, 0.3, 1],
        onUpdate: (value) => {
          element.style.setProperty("--start", String(value));
        },
      });
    });
  },
  [inactiveZone, proximity, movementDuration]
);

useEffect(() => {
  if (disabled) return;

  const handleScroll = () => handleMove();
  const handlePointerMove = (e: PointerEvent) => handleMove(e);

  window.addEventListener("scroll", handleScroll, { passive: true });
  document.body.addEventListener("pointermove", handlePointerMove, {
    passive: true,
  });

  return () => {
    if (animationFrameRef.current) {
      cancelAnimationFrame(animationFrameRef.current);
```

```
      }
      window.removeEventListener("scroll", handleScroll);
      document.body.removeEventListener("pointermove", handlePointerMove);
    };
  }, [handleMove, disabled]);

  return (
    <>
      <div
        className={cn(
          "pointer-events-none absolute -inset-px hidden rounded-[inherit] border opacity-0 transition-opacity",
          glow && "opacity-100",
          variant === "white" && "border-white",
          disabled && "!block"
        )}
      />
      <div
        ref={containerRef}
        style={
          {
            "--blur": `${blur}px`,
            "--spread": spread,
            "--start": "0",
            "--active": "0",
            "--glowingeffect-border-width": `${borderWidth}px`,
            "--repeating-conic-gradient-times": "5",
            "--gradient":
```

```
        variant === "white"
          ? `repeating-conic-gradient(
          from 236.84deg at 50% 50%,
          var(--black),
          var(--black) calc(25% / var(--repeating-conic-gradient-times))
        )`
          : `radial-gradient(circle, #dd7bbb 10%, #dd7bbb00 20%),
        radial-gradient(circle at 40% 40%, #d79f1e 5%, #d79f1e00 15%),
        radial-gradient(circle at 60% 60%, #5a922c 10%, #5a922c00 20%),
        radial-gradient(circle at 40% 60%, #4c7894 10%, #4c789400 20%),
        repeating-conic-gradient(
          from 236.84deg at 50% 50%,
          #dd7bbb 0%,
          #d79f1e calc(25% / var(--repeating-conic-gradient-times)),
          #5a922c calc(50% / var(--repeating-conic-gradient-times)),
          #4c7894 calc(75% / var(--repeating-conic-gradient-times)),
          #dd7bbb calc(100% / var(--repeating-conic-gradient-times))
        )`,
      } as React.CSSProperties
    }
    className={cn(
      "pointer-events-none absolute inset-0 rounded-[inherit] opacity-100 transition-
opacity",
      glow && "opacity-100",
      blur > 0 && "blur-[var(--blur)] ",
      className,
      disabled && "!hidden"
    )}
```

```
      >
        <div
          className={cn(
            "glow",
            "rounded-[inherit]",
            'after:content-[""] after:rounded-[inherit] after:absolute after:inset-[calc(-1*var(--glowingeffect-border-width))]',
            "after:[border:var(--glowingeffect-border-width)_solid_transparent]",
            "after:[background:var(--gradient)] after:[background-attachment:fixed]",
            "after:opacity-[var(--active)] after:transition-opacity after:duration-300",
            "after:[mask-clip:padding-box,border-box]",
            "after:[mask-composite:intersect]",
            "after:[mask-image:linear-gradient(#0000,#0000),conic-gradient(from_calc((var(--start)-var(--spread))*1deg),#00000000_0deg,#fff,#00000000_calc(var(--spread)*2deg))]"
          )}
        />
      </div>
    </>
  );
}
);

GlowingEffect.displayName = "GlowingEffect";

export { GlowingEffect };
```